HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Đỗ Bình Minh**
Ngày sinh	**: 24/12/2005**
Số điện thoại	**: 0329279863**
Mã sinh viên	**: B23DCCN542**
Lớp	**: D23CQCN10-B**
Nhóm lớp	**: 11**

Hà Nội – 2026

Mục lục

1.Tóm tắt các công việc đã thực hiện trong tuần 3
2. Chi tiết các công việc đã thực hiện trong tuần 3
2.1 Tính năng đăng ký 3
2.1.1 Xử lý backend 3
2.1.2 Kết quả 7
2.2 Tính năng đăng nhập 8
2.2.1 Xử lý backend 8
2.2.2 Kết quả 10
2.3 Tính năng đăng xuất 11
2.3.1 Xử lý phía backend. 11
2.3.2 Xử lý phía frontend. 11
2.3.3 Kết quả 12
3 Kết quả đạt được 13
4. Khó khăn gặp phải 13
5. Hướng xử lý 13
6. Kế hoạch tuần tiếp theo 14

1.Tóm tắt các công việc đã thực hiện trong tuần

Trong tuần này, sau khi hệ thống đã có thể tiếp nhận dữ liệu từ Frontend, xử lý logic ở Backend và lưu trữ thành công vào cơ sở dữ liệu ở tuần trước, em tiến hành hoàn thiện các chức năng đăng ký, đăng nhập và đăng xuất cho hệ thống.

Cụ thể, em đã xây dựng API đăng ký tài khoản cho company, thực hiện kiểm tra email tồn tại, mã hóa mật khẩu bằng bcryptjs trước khi lưu vào cơ sở dữ liệu. Đối với chức năng đăng nhập, em triển khai kiểm tra thông tin tài khoản, so sánh mật khẩu đã mã hóa và tạo token xác thực bằng JWT, sau đó lưu token vào cookie để duy trì trạng thái đăng nhập. Bên cạnh đó, em cũng hoàn thiện chức năng đăng xuất bằng cách xóa cookie chứa token khi người dùng thực hiện logout.

Kết quả đạt được là hệ thống đã hoàn thiện luồng xác thực người dùng (đăng ký, đăng nhập, đăng xuất), đảm bảo hoạt động ổn định và sẵn sàng mở rộng thêm các chức năng nghiệp vụ trong các tuần tiếp theo.

2. Chi tiết các công việc đã thực hiện trong tuần

2.1 Tính năng đăng ký

2.1.1 Xử lý backend

```
import companyRoutes from "./company.route";
```

Nhập các route từ file company.route để dùng trong file hiện tại.

```
router.use("/company", companyRoutes)
```

Gắn nhóm route đó vào đường dẫn /company, tức là các API trong company.route sẽ bắt đầu bằng /company

```
import { Request, Response } from "express";
import AccountCompany from "../models/account-company.model";
import bcrypt from "bcryptjs";

export const registerPost = async (req: Request, res: Response) => {
  const existAccount = await AccountCompany.findOne({
    email: req.body.email
  });

  if(existAccount) {
    res.json({
      code: "error",
      message: "Email đã tồn tại trong hệ thống!"
    })
    return;
  }

  // Mã hóa mật khẩu
  const salt = await bcrypt.genSalt(10);
  req.body.password = await bcrypt.hash(req.body.password, salt);

  const newAccount = new AccountCompany(req.body);
  await newAccount.save();

  res.json({
    code: "success",
    message: "Đăng ký tài khoản thành công!"
  })
}
```

- **Import**:
  - express: dùng Request, Response
  - AccountCompany: model MongoDB
  - bcryptjs: mã hóa mật khẩu
- **registerPost**:
  → API đăng ký tài khoản
- **findOne**:
  → kiểm tra email đã tồn tại chưa
- **if (existAccount)**:
  → nếu có rồi → trả về lỗi "Email đã tồn tại"
- **bcrypt.genSalt + hash**:
  → mã hóa mật khẩu trước khi lưu
- **new AccountCompany + save**:
  → tạo tài khoản mới và lưu vào database

- **res.json success**:
  → trả về "Đăng ký thành công"

=>API này kiểm tra email → mã hóa password → lưu user → trả kết quả

```
import mongoose from "mongoose";

const schema = new mongoose.Schema(
  {
    companyName: String,
    email: String,
    password: String,
  },
  {
    timestamps: true, // Tự động sinh ra trường createdAt và updatedAt
  }
);

const AccountCompany = mongoose.model('AccountCompany', schema, "accounts-company");

export default AccountCompany;
```

- import mongoose from "mongoose";
  → dùng thư viện để làm việc với MongoDB
- new mongoose.Schema({...})
  → định nghĩa cấu trúc dữ liệu (companyName, email, password)
- timestamps: true
  → tự tạo thêm createdAt và updatedAt
- mongoose.model('AccountCompany', schema, 'accounts-company')
  → tạo model để thao tác DB
  → lưu dữ liệu vào collection "accounts-company"
- export default AccountCompany;
  → xuất model để dùng ở file khác

=>File này định nghĩa bảng (collection) tài khoản công ty trong MongoDB

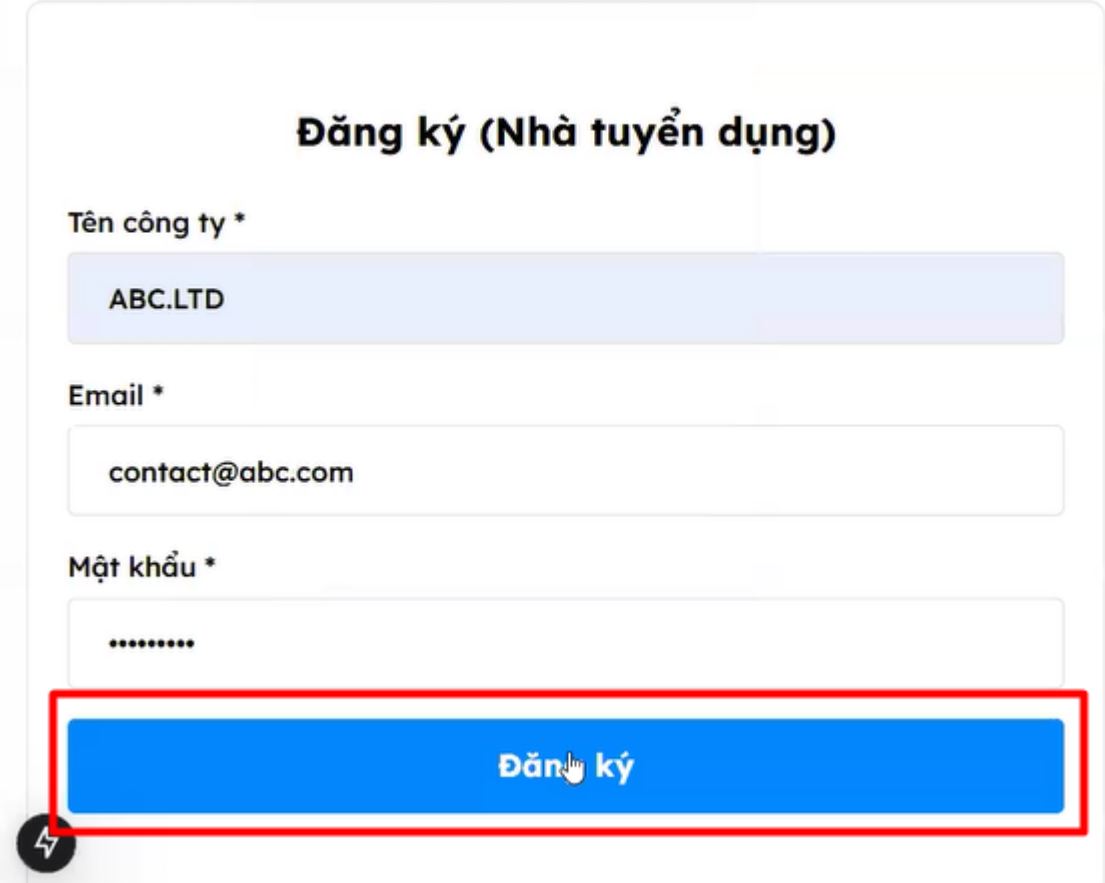
```
import { Router } from "express";
import * as companyController from "../controllers/company.controller";
import * as companyValidate from "../validates/company.validate";

const router = Router();

router.post(
  "/register",
  companyValidate.registerPost,
  companyController.registerPost
)

export default router;
```

- import { Router } from "express";
  → lấy Router để tạo route
- import * as companyController ...
  → nhập các hàm xử lý từ controller
- import * as companyValidate ...
  → nhập các hàm validate dữ liệu
- const router = Router();
  → tạo đối tượng router
- router.post("/register", companyValidate.registerPost, companyController.registerPost)
  → tạo API POST /register
  → chạy **validate trước**, hợp lệ thì mới chạy **controller**
- export default router;
  → xuất router để file khác dùng

=> File này khai báo route đăng ký công ty, kiểm tra dữ liệu trước rồi mới xử lý đăng ký.

2.1.2 Kết quả

Khi ấn đăng ký dữ liệu được lưu thành công vào CSDL trên MongoDB



2.2 Tính năng đăng nhập

2.2.1 Xử lý backend

```
export const loginPost = async (req: Request, res: Response) => {
  const { email, password } = req.body;

  const existAccount = await AccountCompany.findOne({
    email: email
  });

  if(!existAccount) {
    res.json({
      code: "error",
      message: "Email không tồn tại trong hệ thống!"
    })
    return;
  }

  const isPasswordValid = await bcrypt.compare(password, `${existAccount.password}`);

  if(!isPasswordValid) {
    res.json({
      code: "error",
      message: "Mật khẩu không đúng!"
    })
    return;
  }

  const token = jwt.sign(
    {
      id: existAccount.id,
      email: existAccount.email
    },
    `${process.env.JWT_SECRET}`,
    {
      expiresIn: "1d"
    }
  );

  res.cookie("token", token, {
    maxAge: 24 * 60 * 60 * 1000, // 1 ngày
    httpOnly: true,
    secure: process.env.NODE_ENV === "production", // https để true, http để fasle
    sameSite: "lax" // Cho phép gửi cookie giữa các tên miền
  });

  res.json({
    code: "success",
    message: "Đăng nhập thành công!"
```

- **Lấy email, password**
  → từ req.body
- **findOne(email)**
  → kiểm tra tài khoản có tồn tại không
  → không có → trả "Email không tồn tại"
- **bcrypt.compare**
  → so sánh password nhập với password đã mã hóa
  → sai → trả "Mật khẩu không đúng"
- **jwt.sign(...)**
  → tạo token đăng nhập (chứa id, email, hết hạn 1 ngày)
- **res.cookie("token", ...)**
  → lưu token vào cookie trình duyệt
- **res.json success**
  → trả "Đăng nhập thành công"

=>API login: kiểm tra email → kiểm tra mật khẩu → tạo token → lưu cookie → trả kết quả

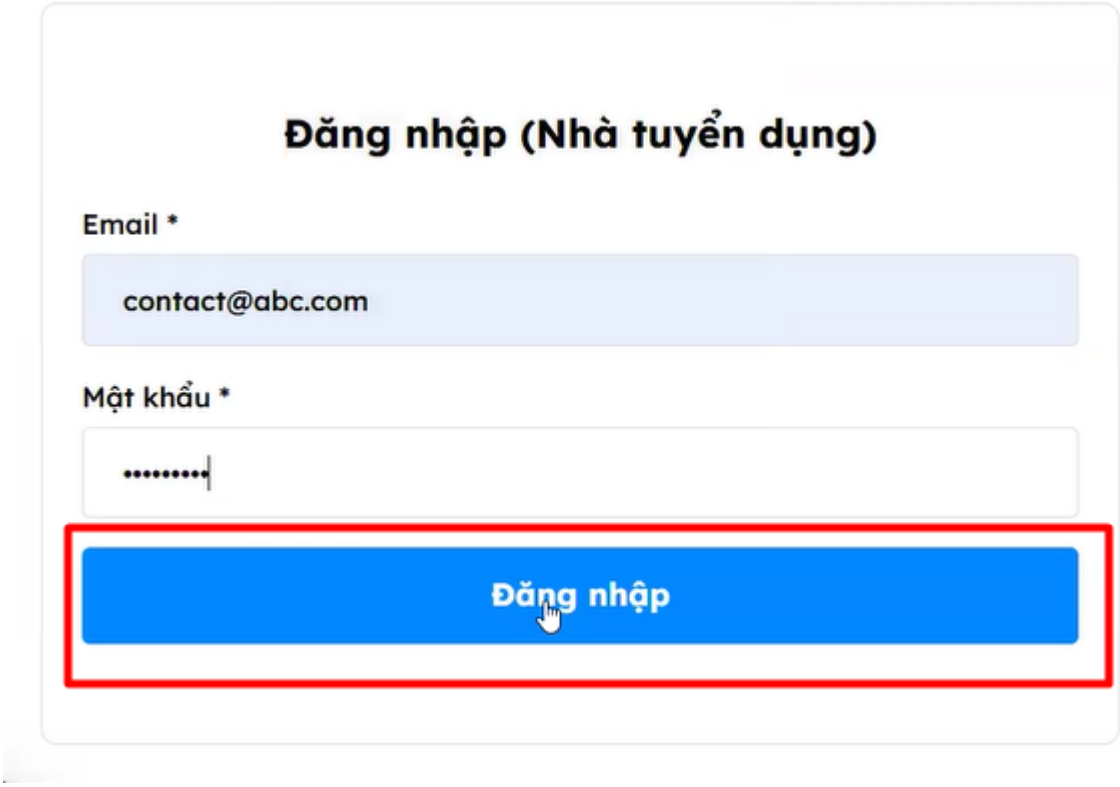
```
+ router.post(
+   "/login",
+   companyValidate.loginPost,
+   companyController.loginPost
+ )
+
```

- router.post("/login", ...)
  → tạo API đăng nhập với phương thức POST
- companyValidate.loginPost
  → kiểm tra dữ liệu gửi lên có hợp lệ không
- companyController.loginPost
  → nếu hợp lệ thì xử lý đăng nhập

=> Dòng này khai báo route đăng nhập, validate trước rồi mới login.

2.2.2 Kết quả

Khi đăng nhập thành công sẽ chuyển đến trang chủ



2.3 Tính năng đăng xuất

2.3.1 Xử lý phía backend.

- Dòng khai báo route

```
+ router.get("/logout", authController.logout)
+
```

- Hàm logout

```
+
+ export const logout = async (req: Request, res: Response) => {
+   res.clearCookie("token");
+   res.json({
+     code: "success",
+     message: "Đã đăng xuất!"
+   })
+ }
```

Khi người dùng gọi API /logout, hệ thống sẽ **xóa cookie chứa token đăng nhập** bằng res.clearCookie("token")

2.3.2 Xử lý phía frontend.

- Bắt sự kiện onClick cho nút đăng xuất

```
-               <li
-                 className="py-[10px] px-[16px] rounded-[4px] flex items-center justify-between hover:bg-
[#000096] relative group/sub-2 cursor-pointer"
-                 onClick={() => handleLogout("/user/login")}
-               >
                  Đăng xuất
```

Khi click vào nút đăng xuất sẽ chạy vào hàm handleLogout và truyền tham số là đường dẫn của trang đăng nhập.

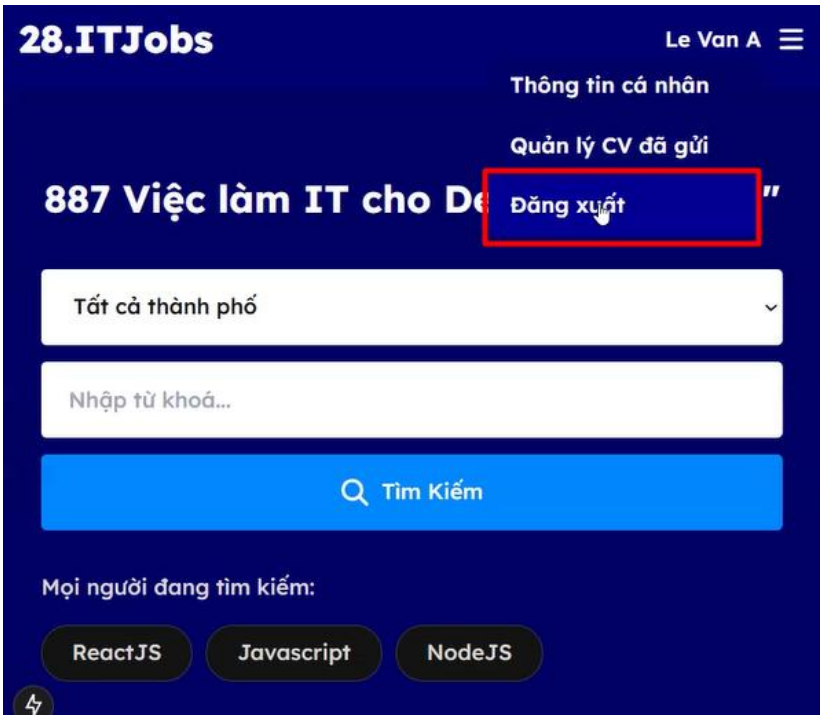
- Hàm handleLogout

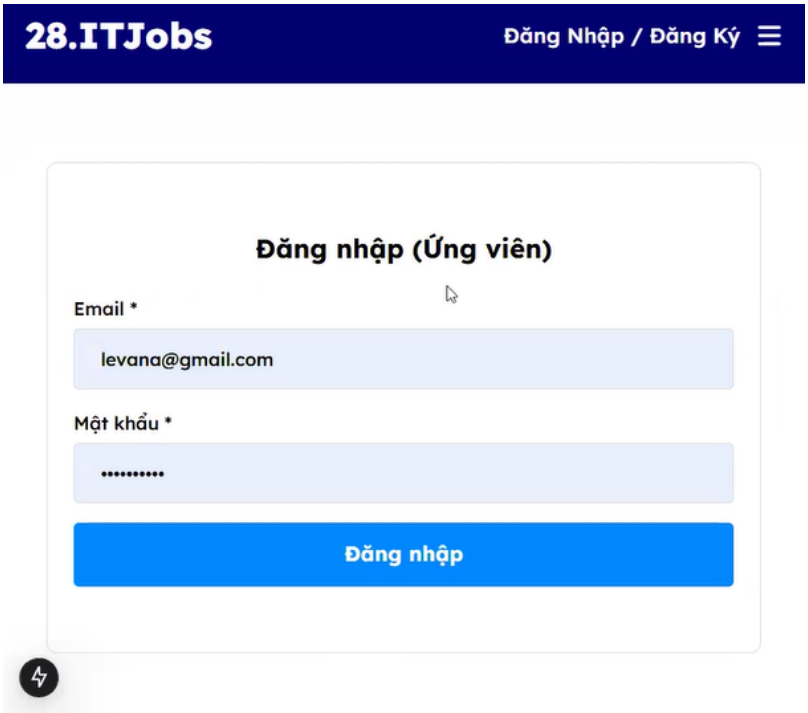
```
const handleLogout = (url: string) => {
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/auth/logout`, {
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "success") {
        router.push(url);
      }
    });
}
```

Đoạn code này dùng để **gọi API đăng xuất từ frontend**. Nó gửi request đến backend kèm theo cookie đăng nhập, sau đó nhận phản hồi từ server. Nếu backend trả về success thì chương trình sẽ **chuyển người dùng sang trang khác** bằng router.push(url). Nói ngắn gọn, tác dụng của đoạn này là **thực hiện đăng xuất và điều hướng lại giao diện sau khi đăng xuất thành công**.

2.3.3 Kết quả

Khi người dùng ấn nút đăng xuất sẽ được quay trở lại trang đăng nhập





3 Kết quả đạt được

- Xây dựng thành công API đăng ký tài khoản cho company
- Xây dựng thành công API đăng nhập sử dụng bcrypt và JWT
- Triển khai chức năng đăng xuất bằng cách xóa cookie token
- Kết nối MongoDB thành công và lưu trữ dữ liệu tài khoản công ty
- Mã hóa mật khẩu trước khi lưu vào database đảm bảo an toàn

4. Khó khăn gặp phải

- Gặp lỗi khi kiểm tra email tồn tại do chưa hiểu rõ cách dùng findOne trong MongoDB
- Khó khăn trong việc sử dụng bcrypt.compare để so sánh mật khẩu đã mã hóa
- Lỗi khi tạo token JWT do chưa cấu hình đúng biến môi trường JWT_SECRET
- Chưa hiểu rõ cơ chế cookie nên gặp lỗi khi lưu và xóa token
- Frontend không nhận được cookie khi gọi API do thiếu cấu hình credentials
- Việc tổ chức code theo controller – route – model ban đầu còn bị nhầm lẫn

5. Hướng xử lý

- Đối với kiểm tra email:

Sử dụng AccountCompany.findOne({ email }) để tìm tài khoản theo email trong database

• Đối với mã hóa và kiểm tra mật khẩu:
Sử dụng bcrypt.genSalt và bcrypt.hash để mã hóa, bcrypt.compare để so sánh mật khẩu

• Đối với JWT:
Cấu hình biến môi trường JWT_SECRET trong file .env và sử dụng jwt.sign() để tạo token

• Đối với cookie:
Sử dụng res.cookie() để lưu token và res.clearCookie() để xóa khi logout

• Đối với frontend:
Thêm cấu hình credentials: "include" khi gọi API để gửi kèm cookie

• Đối với cấu trúc project:
Tách rõ các phần route, controller, model, validate để code dễ quản lý và mở rộng

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục phát triển các chức năng liên quan đến quản lý thông tin người dùng và nhà tuyển dụng trên hệ thống.

Cụ thể:

• Xây dựng **trang thông tin cá nhân cho ứng viên**, cho phép người dùng xem và cập nhật các thông tin cá nhân như họ tên, email, số điện thoại, v.v.

• Kết nối dữ liệu giữa frontend và backend để hiển thị thông tin cá nhân ứng viên từ cơ sở dữ liệu MongoDB.

• Xây dựng **trang thông tin công ty cho nhà tuyển dụng**, cho phép hiển thị và chỉnh sửa thông tin công ty.

• Thiết kế giao diện hiển thị thông tin phù hợp với từng loại tài khoản (ứng viên / nhà tuyển dụng).

• Triển khai API backend để lấy và cập nhật dữ liệu thông tin người dùng và công ty.